



North South University

Department of Electrical & Computer Engineering

Assignment-03

Faculty initial: smh2

Course Code: 331

Course Title:

Microprocessor Interfacing and
Embedded Systems

Submission Date:22/10/2024

Name: ISRAT ZAMAN SRITY

ID: 2211084042

Section: 8

Swapping Two Numbers Using a Function with Stack ensures that the original value of `r4` is preserved when the function returns:

Code:

```
.section .text
.global _start
_start:
    MOV R0, #3    // Load 3 into r0
    MOV R1, #4    // Load 4 into r1
    MOV R4, #50   // Set R4 to 50 will be preserved(callee)

    BL swap      // Branch to swap function
                //After swapping, r0 = 4 and r1 = 3
    B end        //Jump to the end label
swap:
    PUSH {R4, LR} // Save R4 and LR in stack

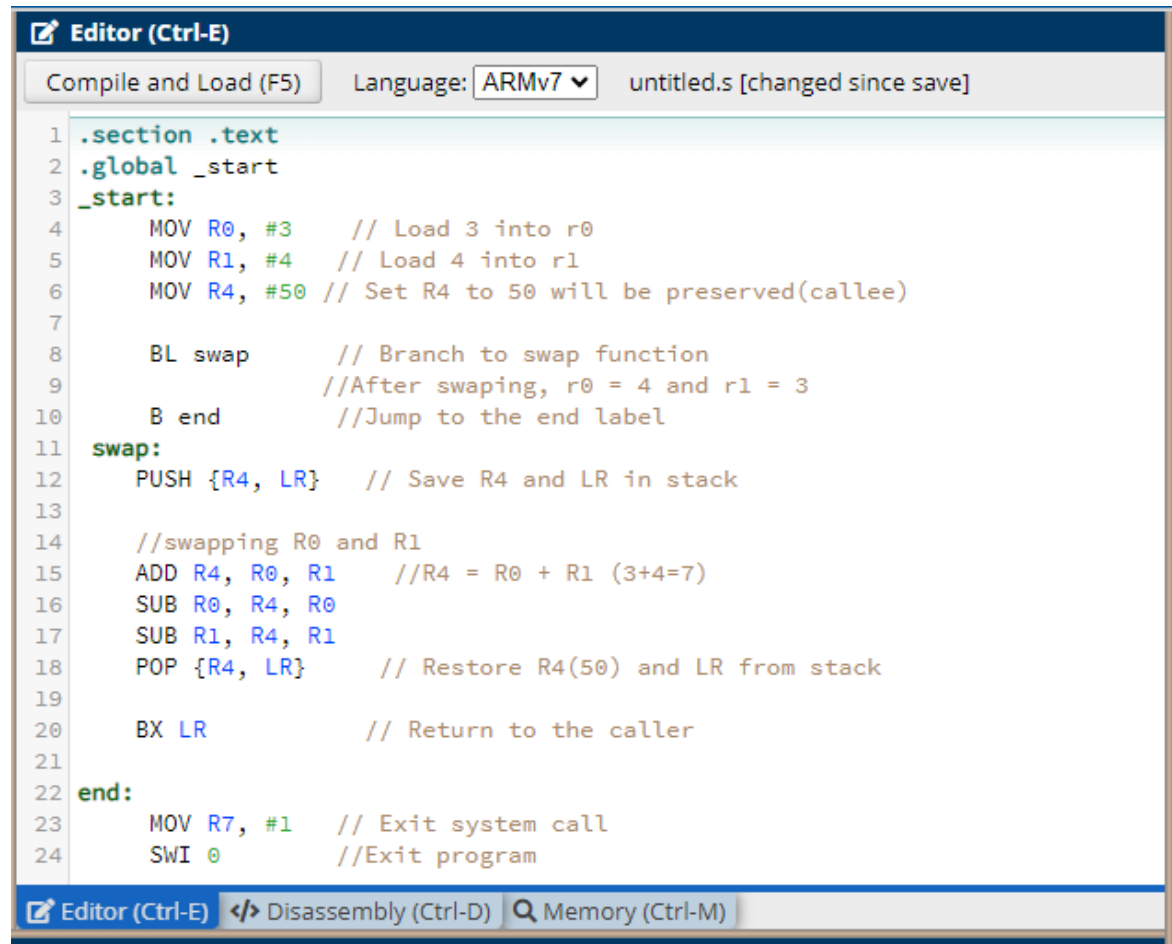
    //swapping R0 and R1
    ADD R4, R0, R1 //R4 = R0 + R1 (3+4=7)
    SUB R0, R4, R0
    SUB R1, R4, R1
    POP {R4, LR}  // Restore R4(50) and LR from stack

    BX LR        // Return to the caller

end:
    MOV R7, #1    // Exit system call
    SWI 0         //Exit program
```

Snapshot:

Code :

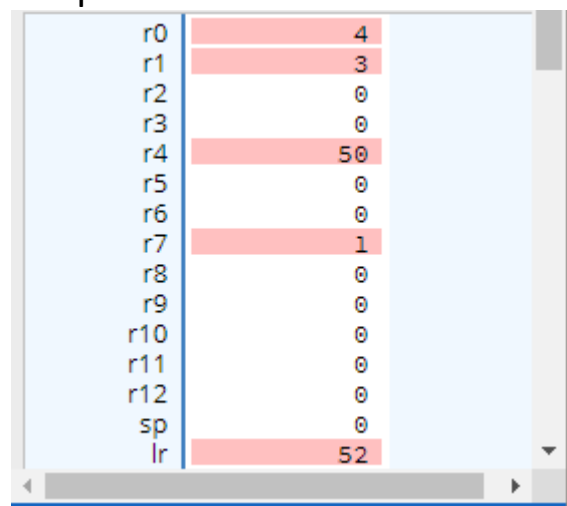


The screenshot shows an ARMv7 assembly code editor with the following code:

```
1 .section .text
2 .global _start
3 _start:
4     MOV R0, #3    // Load 3 into r0
5     MOV R1, #4    // Load 4 into r1
6     MOV R4, #50   // Set R4 to 50 will be preserved(callee)
7
8     BL swap       // Branch to swap function
9                 //After swaping, r0 = 4 and r1 = 3
10    B end          //Jump to the end label
11 swap:
12    PUSH {R4, LR}  // Save R4 and LR in stack
13
14    //swapping R0 and R1
15    ADD R4, R0, R1  //R4 = R0 + R1 (3+4=7)
16    SUB R0, R4, R0
17    SUB R1, R4, R1
18    POP {R4, LR}   // Restore R4(50) and LR from stack
19
20    BX LR          // Return to the caller
21
22 end:
23    MOV R7, #1     // Exit system call
24    SWI 0          //Exit program
```

The editor interface includes a toolbar with 'Compile and Load (F5)', 'Language: ARMv7', and 'untitled.s [changed since save]'. At the bottom, there are tabs for 'Editor (Ctrl-E)', 'Disassembly (Ctrl-D)', and 'Memory (Ctrl-M)'.

Output:

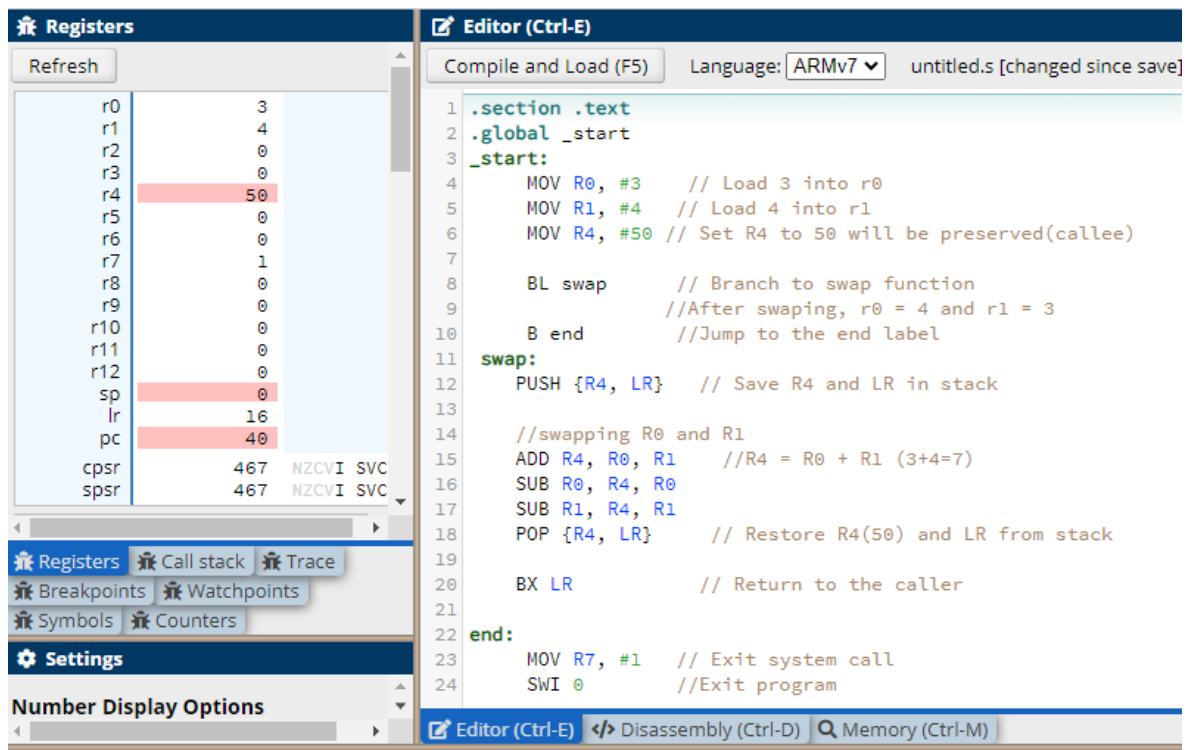


r0	4
r1	3
r2	0
r3	0
r4	50
r5	0
r6	0
r7	1
r8	0
r9	0
r10	0
r11	0
r12	0
sp	0
lr	52

Explanation:

1. Before Modifying Registers:

- At the beginning of the code, before any modifications or swaps happen, the registers are loaded with values:
 - R0 = 3 (set by MOV R0, #3)
 - R1 = 4 (set by MOV R1, #4)
 - R4 = 50 (set by MOV R4, #50, to be preserved)

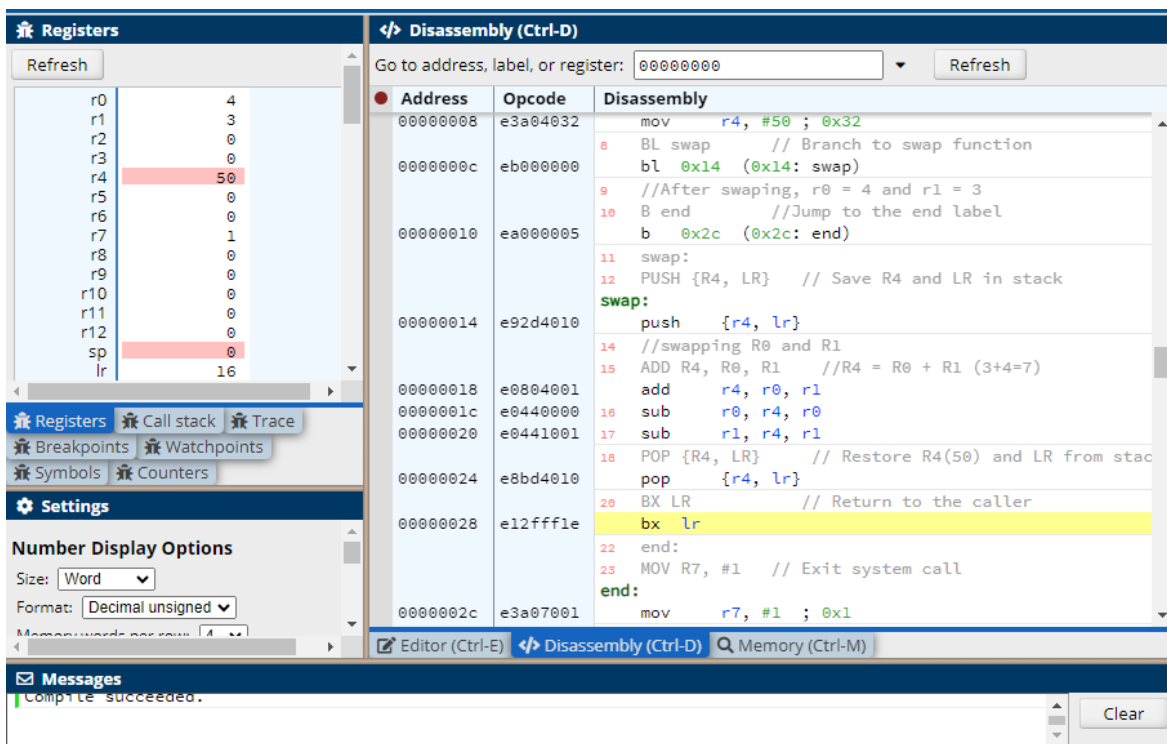


Here,

- . R4 is a callee-saved register, meaning its value must be preserved throughout the function call.
- . The swap function will modify some registers, but it will also ensure that r4 returns to its original value when the function is done.

2. After the Swap:

- The swap function is called using the BL swap instruction.
- Inside the swap function:
 - The current values of r4 and the link register (LR) are pushed onto the stack using PUSH {R4, LR}. This ensures that the original value of r4 (which is 50) and the return address (stored in LR) are saved.
 - A swap between R0 and R1 is performed using ADD and SUB instructions:
 - ADD R4, R0, R1 computes the sum of r0 and r1 and stores it in r4. So, $r4 = 3 + 4 = 7$.
 - SUB R0, R4, R0 swaps the value of r0 by computing $r4 - r0$, i.e., $r0 = 7 - 3 = 4$.
 - SUB R1, R4, R1 swaps the value of r1 by computing $r4 - r1$, i.e., $r1 = 7 - 4 = 3$.
 - The original value of r4 (50) is restored from the stack using POP {R4, LR}, so r4 becomes 50 again.
 - The BX LR instruction branches back to the caller (the instruction after the BL swap).



3. Ensuring the Original Value of r4 is Preserved:

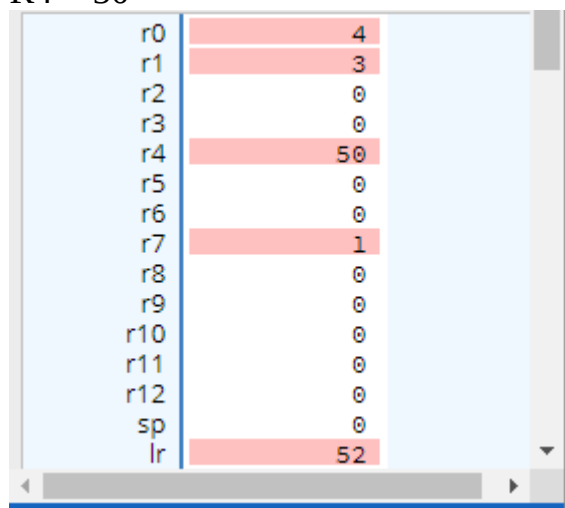
- To preserving the original value of R4 is the ,”PUSH {R4, LR}” .
- At the start of the swap function and POP {R4, LR} at the end of the function.
- By pushing R4 to the stack before modifying it, the original value (50) is saved, and after the swap is completed, the value is restored using the POP instruction.
- This ensures that even though R4 is used temporarily in the swap function, the original value of 50 is retained when the function returns.

Final Register Values:

- **After the swap:**
 - R0 = 4 (swapped from r1)
 - R1 = 3 (swapped from r0)
 - R4 = 50 (restored after the swap, as it was saved and popped from the stack)

Outputs:

- R0 = 4
- R1 = 3
- R4 = 50



r0	4
r1	3
r2	0
r3	0
r4	50
r5	0
r6	0
r7	1
r8	0
r9	0
r10	0
r11	0
r12	0
sp	0
lr	52